CS7643 Assignment 2

Tyler Crosse
Georgia Tech
tcrosse3@gatech.edu

1 Theory Problem Set

1.1 Theory Q1

The convolution layer inside of a CNN is intrinsically an affine transformation: A vector is received as input and is multiplied with a matrix to produce an output (to which a bias vector is usually added before passing the result through a nonlinearity). This operation can be represented as $y = Ax$, in which A describes the affine transformation.

We will first revisit the convolution layer as discussed in the class. Consider a convolution layer with a 3×3 kernel W, operated on a single input channel X, represented as:

$$W = \begin{bmatrix} w_{(0,0)}, w_{(0,1)}, w_{(0,2)} \\ w_{(1,0)}, w_{(1,1)}, w_{(1,2)} \\ w_{(2,0)}, w_{(2,1)}, w_{(2,2)} \end{bmatrix}, X = \begin{bmatrix} x_{(0,0)}, x_{(0,1)}, x_{(0,2)} \\ x_{(1,0)}, x_{(1,1)}, x_{(1,2)} \\ x_{(2,0)}, x_{(2,1)}, x_{(2,2)} \end{bmatrix}$$

Using this example, let us work out a **stride-4** convolution layer, with **zero padding size of 2** and let the output be Y. Consider 'flattening' the input tensor X in row-major order as:

$$x = \left[x_{(0,0)}, x_{(0,1)}, x_{(0,2)}, \ldots, x_{(2,0)}, x_{(2,1)}, x_{(2,2)}\right]^\top$$

Note this x does not and should not include any padding! Write down this convolution of W and X as a matrix operation A such that: $y = Ax$. This is similar to the im2col operation in the lectures but slightly different. Output y is also flattened in row-major order. NOTE: For this problem we are referring to a convolution in the deep learning context (i.e. do not flip the kernel). Note that the flattened x does not contain any padding.

(a) What are the dimensions of A?
(b) Write down the entries in this matrix A.

1.1.1 (a)

Output size for square input X and kernel W:

$$
\begin{aligned}
\text{Output Height} &= \text{Output Width} \\
&= \left\lfloor \frac{H_{\text{in}} - K_{\text{height}} + 2\,P_h}{S} \right\rfloor + 1 \\
&= \left\lfloor \frac{3 - 3 + 2(2)}{4} \right\rfloor + 1 \\
&= \left\lfloor \frac{4}{4} \right\rfloor + 1 \\
&= 2
\end{aligned}
$$

Where:

- H_{in} and W_{in} are the input height and width (respectively), 3 here.
- K_{height} and K_{width} are the kernel (filter) height and width, 3 here.
- P is the padding applied to both sides in each dimension, 2 here.
- S is the stride, 4 here.

The shape of the output tensor Y is:

$$
(\text{Batch Size},\ \text{Output Height},\ \text{Output Width},\ C_{\text{out}}) = (1, 2, 2, 1)
$$

From the definition of matrix multiplication, if A is an $m \times n$ matrix and x^T is an $n \times 1$ column vector, then the product Ax is an $m \times 1$ column vector.

Flattening Y (2×2) means the vector y has a length m of 4 and a shape 1×4. Flattening the input X means the vector x (3×3) has a length n of 9 and a shape 1×9.

Substituting, **the dimensions of A are** 4×9

1.1.2 (b)

Figure 1 shows a visual representation of sliding the kernel W across the input X to form Y. The padding of 2 and stride of 4 mean that only four convolutions happen and each one only uses one value of X and one value of W. This corresponds to the 2×2 output shape of Y found above in (a).

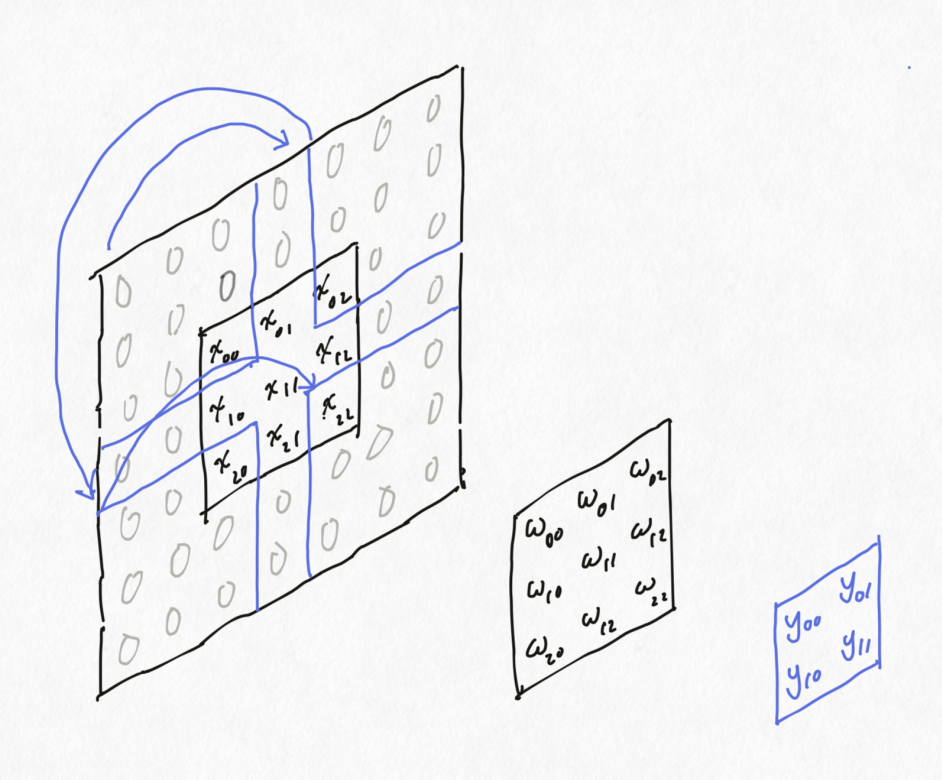


Figure 1: W striding across X to create Y

In the first convolution, the first element of X and the last element of W are used to form the first element of Y. The same process can be followed to find which elements of W and X correspond to the elements of Y: $y_{(0,0)} = w_{(2,2)} \cdot x_{(0,0)}$ $y_{(0,1)} = w_{(2,0)} \cdot x_{(0,2)}$ $y_{(1,0)} = w_{(0,2)} \cdot x_{(2,0)}$ $y_{(1,1)} = w_{(0,0)} \cdot x_{(2,2)}$

Mapping the matrix indices of X to the positions in the flattened x vector:

$x_{(0,0)} = x_0$ $x_{(0,2)} = x_2$ $x_{(2,0)} = x_6$ $x_{(2,2)} = x_8$

This shows that only the 1st, 3rd, 7th, and 9th elements of x are used to create Y. In the matrix multiply, these correspond to the 1st, 3rd, 7th, and 9th columns of A. And because of the zero padding, the remaining elements of A are zero. This gives the entries of the matrix A:

$$A = \begin{bmatrix} w_{(2,2)} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & w_{(2,0)} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & w_{(0,2)} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & w_{(0,0)} \end{bmatrix}.$$

1.2 Theory Q2

Consider a specific 2 hidden layer ReLU network with inputs $x \in \mathbb{R}, 1$ dimensional outputs, and 2 neurons per hidden layer. This function is given by

$$h(x) = W^{(3)} \max\left\{0, W^{(2)} \max\left\{0, W^{(1)}x + b^{(1)}\right\} + b^{(2)}\right\} + b^{(3)}$$

where the max is element-wise, with weights:

$$\begin{aligned} W^{(1)} &= \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} \\ b^{(1)} &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ W^{(2)} &= \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \\ b^{(2)} &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ W^{(3)} &= \begin{bmatrix} 1 & 1 \end{bmatrix} \\ b^{(3)} &= -1 \end{aligned}$$

An interesting property of networks with piece-wise linear activations like the ReLU is that on the whole they compute piece-wise linear functions. At each of the following points $x = x_o$, determine the value of the new weight $W \in \mathbb{R}$ and bias $b \in \mathbb{R}$ such that $\frac{dh(x)}{dx}\Big|_{x=x_0} = W$ and $Wx_o + b = h(x_o)$.

$$\begin{aligned} x_o &= 2 \\ x_o &= -1 \\ x_o &= 1 \end{aligned}$$

Splitting it into layers:

$$\begin{aligned} z^{(1)} &= W^{(1)}x + b^{(1)} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ \text{layer } 1 = a^{(1)} &= \max\left\{0, z^{(1)}\right\} \\ z^{(2)} &= W^{(2)}a^{(1)} + b^{(2)} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} a^{(1)} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ \text{layer } 2 = a^{(2)} &= \max\left\{0, z^{(2)}\right\} \\ \text{output} = h(x) &= W^{(3)}a^{(2)} + b^{(3)} = \begin{bmatrix} 1 & 1 \end{bmatrix} a^{(2)} + (-1) \end{aligned}$$

Working backward using the chain rule for the individual pieces,

$$\frac{dh(x)}{dx} = W^{(3)} \frac{da^{(2)}}{dx}$$

$$\frac{da^{(2)}}{dx} = \begin{bmatrix} \mathbb{1}_{z^{(2)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(2)}>0} \end{bmatrix} \frac{dz^{(2)}}{dx}$$

Where $\mathbb{1}$ is the indicator function.

$$\frac{dz^{(2)}}{dx} = W^{(2)} \frac{da^{(1)}}{dx}$$

$$\frac{da^{(1)}}{dx} = \begin{bmatrix} \mathbb{1}_{z^{(1)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(1)}>0} \end{bmatrix} \frac{dz^{(1)}}{dx}$$

$$\frac{dz^{(1)}}{dx} = W^{(1)}$$

Putting it together gives the full symbolic derivative.

$$\frac{dh(x)}{dx} = W^{(3)} \begin{bmatrix} \mathbb{1}_{z^{(2)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(2)}>0} \end{bmatrix} W^{(2)} \begin{bmatrix} \mathbb{1}_{z^{(1)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(1)}>0} \end{bmatrix} W^{(1)}$$

And substituting for W

$$\frac{dh(x)}{dx} = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} \mathbb{1}_{z^{(2)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(2)}>0} \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} \mathbb{1}_{z^{(1)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(1)}>0} \end{bmatrix} \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}$$

Evaluating at $x_0 = 2$

$$z^{(1)} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} (2) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

$$\text{layer } 1 = a^{(1)} = \max\left\{0, z^{(1)}\right\} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

$$z^{(2)} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 7 \\ 9 \end{bmatrix}$$

$$\text{layer } 2 = a^{(2)} = \max\left\{0, z^{(2)}\right\} = \begin{bmatrix} 7 \\ 9 \end{bmatrix}$$

$$\text{output} = h(2) = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 7 \\ 9 \end{bmatrix} + (-1) = 15$$

Referencing the symbolic derivative

$$\frac{dh(x)}{dx} = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} \mathbb{1}_{z^{(2)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(2)}>0} \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} \mathbb{1}_{z^{(1)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(1)}>0} \end{bmatrix} \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}$$

All values of $z^{(2)}$ and $z^{(1)}$ are positive.

$$\frac{dh(x)}{dx}\Big|_{x=2} = W = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} = 6$$

Plugging this into $Wx_0 + b = h(x_0)$ to find b

$$6(2) + b = 15 \implies 12 + b = 15 \implies b = 3$$

Evaluating at $x_0 = -1$

$$z^{(1)} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}(-1) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix}$$

$$\text{layer } 1 = a^{(1)} = \max\left\{0, z^{(1)}\right\} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

$$z^{(2)} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} 0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1.5 \end{bmatrix}$$

$$\text{layer } 2 = a^{(2)} = \max\left\{0, z^{(2)}\right\} = \begin{bmatrix} 1 \\ 1.5 \end{bmatrix}$$

$$\text{output} = h(-1) = \begin{bmatrix} 1 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 1.5 \end{bmatrix} + (-1) = 1.5$$

Referencing the symbolic derivative

$$\frac{dh(x)}{dx} = \begin{bmatrix} 1 & 1 \end{bmatrix}\begin{bmatrix} \mathbb{1}_{z^{(2)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(2)}>0} \end{bmatrix}\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} \mathbb{1}_{z^{(1)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(1)}>0} \end{bmatrix}\begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}$$

Both parts of $z^{(2)}$ are positive but only the second element of $z^{(1)}$ is positive.

$$\frac{dh(x)}{dx}\Big|_{x=-1} = W = \begin{bmatrix} 1 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} = 1.5$$

Plugging this into $Wx_0 + b = h(x_0)$ to find b

$$1.5(-1) + b = 1.5 \implies -1.5 + b = 1.5 \implies b = 3$$

Evaluating at $x_0 = 1$

$$z^{(1)} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}(1) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}$$

$$\text{layer } 1 = a^{(1)} = \max\left\{0, z^{(1)}\right\} = \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}$$

$$z^{(2)} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4.5 \\ 5.5 \end{bmatrix}$$

$$\text{layer } 2 = a^{(2)} = \max\left\{0, z^{(2)}\right\} = \begin{bmatrix} 4.5 \\ 5.5 \end{bmatrix}$$

$$\text{output} = h(-1) = \begin{bmatrix} 1 & 1 \end{bmatrix}\begin{bmatrix} 4.5 \\ 5.5 \end{bmatrix} + (-1) = 9$$

Referencing the symbolic derivative

$$\frac{dh(x)}{dx} = \begin{bmatrix} 1 & 1 \end{bmatrix}\begin{bmatrix} \mathbb{1}_{z^{(2)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(2)}>0} \end{bmatrix}\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} \mathbb{1}_{z^{(1)}>0} & 0 \\ 0 & \mathbb{1}_{z^{(1)}>0} \end{bmatrix}\begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}$$

Both parts of $z^{(2)}$ and $z^{(1)}$ are positive.

$$\frac{dh(x)}{dx}\Big|_{x=1} = W = \begin{bmatrix} 1 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} = 6$$

Plugging this into $Wx_0 + b = h(x_0)$ to find b

$$6(1) + b = 9 \implies 6 + b = 9 \implies b = 3$$

To summarize all the annoying error prone work above:

- $x_0 = 2$, $W = 6$, $b = 3$
- $x_0 = -1$, $W = 1.5$, $b = 3$
- $x_0 = 1$, $W = 6$, $b = 3$

Thank god for computers!

1.3 Theory Q3

The ReLU function sometimes has problems where it creates "deadneurons" due to producing 0 gradients when the activation is less than 0. In practice, however, this does not seem to be a big problem. Why is this?

A handful of factors can help mitigate the issue.

- Proper initialization can help prevent the neurons from dying. Techniques like He initialization help maintain the variance of activations across layers [1, 2].
- Adaptive learning rates can help reduce large changes in weights that can kill neurons. Gradual and controlled updates reduce the risk of neurons dying.
- Models often have excess capacity and can tolerate some neurons dying.

2 Paper Review

ImageNet-trained CNNs are Biased Towards Texture; Increasing Shape Bias Improves Accuracy and Robustness by Geirhos et al. [3]

2.1 Paper Q1

What is the main contribution of this paper? In other words, briefly summarize its key insights. What are some strengths and weaknesses of this paper?

This paper challenges the common belief that Convolutional Neural Networks (CNNs) recognize objects primarily based on shape. Instead, it provides strong empirical evidence that ImageNet-trained CNNs rely heavily on texture cues rather than global shape features, a fundamental departure from human perception. Through experiments with "texture-shape cue conflict images" [3], the study shows that CNNs classify objects by their texture, whereas humans focus on shape. The authors introduce Stylized-ImageNet (SIN), a dataset that removes informative textures using style transfer, forcing CNNs to develop a shape-based representation. Training on SIN improves model robustness to image distortions and adversarial attacks while enhancing generalization to unseen datasets. Strengths of the paper include its well-controlled psychophysical experiments and practical impact on improving CNN robustness. However, a limitation is that the study focuses primarily on ResNet-50, leaving open questions about whether other architectures (like transformers) exhibit similar biases.

2.2 Paper Q2

What is your personal takeaway from this paper? This could be expressed either in terms of your perceived novelty of this paper compared to others you've read in the field, potential future directions of research in the area that the authors haven't addressed, or anything else that struck you as being noteworthy.

This paper presents a convincing argument that CNNs do not inherently learn shape-based representations but do so based on training data bias. The results suggest that ImageNet's texture-heavy nature encourages shortcut learning, which could explain CNN vulnerability to domain shifts and adversarial perturbations. One of the most interesting aspects is that removing texture cues through SIN improves generalization without requiring changes to the model architecture. Future research could explore whether shape-biased models improve performance in real-world applications such as medical imaging, robotics, or self-driving cars, where robustness is crucial. Additionally, it would be valuable to investigate how different architectures (like Vision Transformers) compare in terms of texture vs. shape bias. The findings also raise philosophical questions: should CNNs mirror human perception, or should they develop their own optimal strategies? This paper motivates more general questions about how the data they are trained on informs the representations they create and the biases they exhibit.

2.3 Paper Q3

Why would we care about the biases of the neural network, even if it can do well on the training set? Should we desire the network to have the same biases as humans or not? Why or why not?

Biases in neural networks matter because they affect generalization, interpretability, and real-world reliability. A model that overfits to textures rather than learning robust shape representations may fail when encountering noisy, occluded, or out-of-distribution data. This is particularly concerning in safety-critical applications, such as medical diagnosis, where texture bias could lead to incorrect classifications of tumors. Is it always good for CNNs to mimic human biases? In some cases, yes—human-like shape perception leads to greater robustness and interpretability. However, human perception is not always optimal. We are prone to optical illusions and cognitive biases that a machine might avoid. The ideal approach is to develop flexible models that can adapt their biases based on the task. For some applications, human-like perception may be beneficial, while for others, a purely data-driven approach may yield superhuman performance. Striking the right balance is crucial for building AI systems that are both powerful and trustworthy.

2.4 Paper Q4

Why would training on stylized images change the bias of the network, and why might this bias generalize better to datasets with corruptions?

Training on Stylized-ImageNet (SIN) alters CNN biases by removing texture as a reliable feature, forcing the network to rely on global shape information. Standard CNNs learn highly localized features, such as fur patterns or surface textures, because these provide strong class-discriminative signals in ImageNet. However, this reliance on texture makes them fragile to real-world distortions, as noise, compression artifacts, or adversarial perturbations can easily disrupt texture features. Shape, in contrast, is a more stable visual cue that remains relatively consistent under transformations. Models trained on SIN show greater robustness to noise, blurring, and other corruptions because they prioritize global structure over local details. Empirical results confirm that SIN-trained CNNs outperform standard ImageNet models on corrupted datasets and adversarial robustness benchmarks. This suggests that carefully curating training data can significantly improve model generalization, making AI systems more resilient and reliable in real-world environments.

3 Writeup

3.1 Part 1 ConvNet

Put your training curve here:

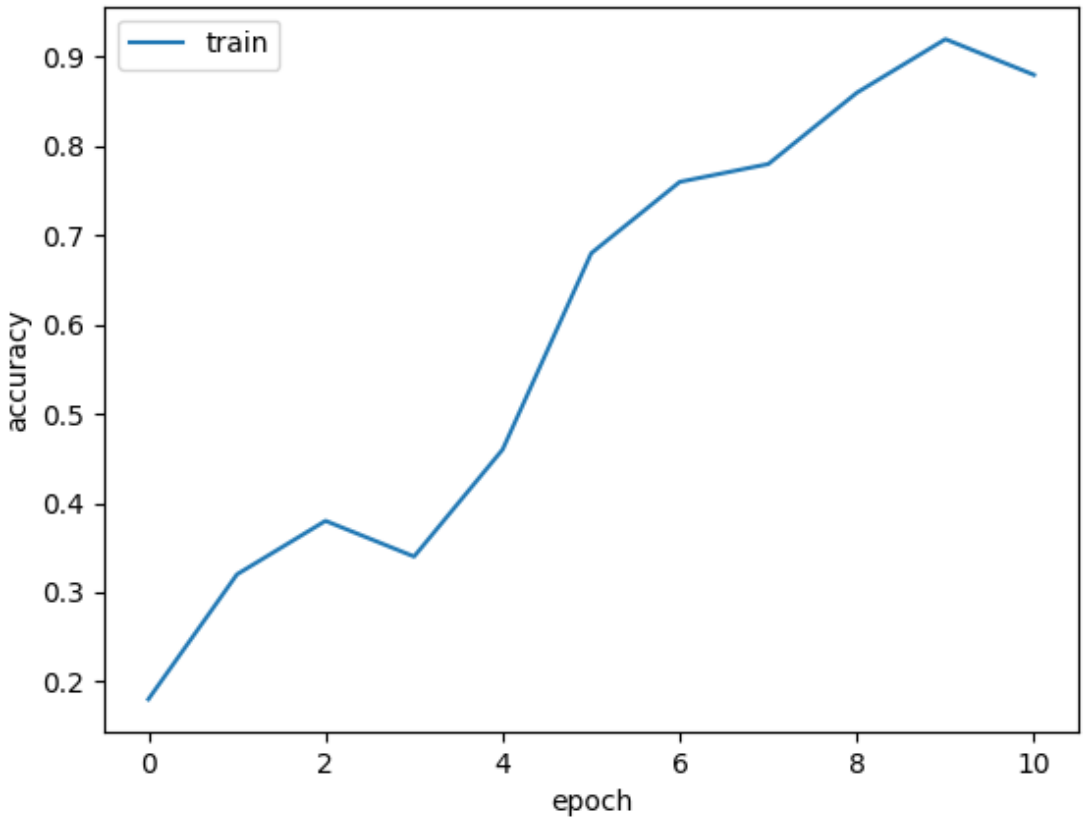


Figure 2: Part 1 ConvNet training curve

3.2 My CNN Model

3.2.1 Model Design

Describe and justify your model design in plain text here:

The main inspiration for my model came from VGG [4, 2, 5]. It consists of three convolution blocks, each with two 3x3 convolution layers with ReLU followed by a max pooling. Two 3x3 convolutions have the same receptive field as a single 5x5 conv, but have fewer parameters and take less computation. After each convolution layer within the block, there is a batch norm that is intended to stabilize and normalize the output of the convolution layer right before it [2]. The channels progressively increase after each block from 3 -¿ 64 -¿ 128 -¿ 256, which is intended to allow the network to capture hierarchical representations in the input image, in a similar manner to VGG. After the three convolution blocks, there is a classifier that consists of two fully connected layers with a dropout, which was added to help avoid overfitting.

Breaking down the parameter count a bit: The first conv block:

- First Conv2d: $(3 \times 3 \times 3 \times 64) + 64 = 1,792$
- First BatchNorm2d: $2 \times 64 = 128$
- Second Conv2d: $(3 \times 3 \times 64 \times 64) + 64 = 36,928$
- Second BatchNorm2d: $2 \times 64 = 128$

The second conv bock:

- First Conv2d: $(3 \times 3 \times 64 \times 128) + 128 = 73,856$
- First BatchNorm2d: $2 \times 128 = 256$
- Second Conv2d: $(3 \times 3 \times 128 \times 128) + 128 = 147,584$
- Second BatchNorm2d: $2 \times 128 = 256$

The third conv block:

- First Conv2d: $(3 \times 3 \times 128 \times 256) + 256 = 295,168$
- First BatchNorm2d: $2 \times 256 = 512$
- Second Conv2d: $(3 \times 3 \times 256 \times 256) + 256 = 590,080$
- Second BatchNorm2d: $2 \times 256 = 512$

The final classifier

- First Linear: $(256 \times 4 \times 4 \times 512) + 512 = 2,097,664$
- BatchNorm1d: $2 \times 512 = 1,024$
- Second Linear: $(512 \times 10) + 10 = 5,130$

In total, there are 3,250,998 parameters, most of which (≈2M) are in the first fully connected layer of the classifier.

3.2.2 Hyper-parameters

Describe and justify your choice of hyper-parameters:

The model with the best final accuracy was trained with the following hyper-parameters

- Batch Size: 256
- Learning Rate: 0.01639
- Regularization: 0.000002787
- Epochs: 10
- Steps: [6, 8]
- Warmup: 2
- Momentum 0.8747
- Gamma: 0.3735
- Imbalance: Regular
- Loss: Cross Entropy

These were found first with some manual effort of 5 trials and then fine-tuned with a Bayesian search using Optuna on an RTX4090 RunPod instance with an additional 20 trials. Learning rate, followed by regularization, had the largest impact on the final model accuracy. Batch normalization allows for a higher learning rate (0.016 vs 0.0001), and the 2 epoch warmup helps stabilize this higher rate during the initial portion of the training. The BatchNorm, Dropout, and MaxPooling also provide implicit regularization, which allows the regularization value used to be quite small (2.78e-6 vs 0.0005) [5]. The smaller regularization decays the weights less allowing them to grow larger. The learning rate schedule complements these choices with a 2-epoch warmup period to stabilize early training, followed by aggressive decay (gamma: 0.37 vs 1.0) at epochs 6 and 8 for fine-tuning. The slightly lower momentum (0.87 vs 0.9) helps prevent overshooting with the higher learning rate.

3.2.3 Final Accuracy

What's your final accuracy on the validation set?

The best precision @1 Accuracy was 0.8737.

3.3 Data Wrangling

3.3.1 Regular CE loss on imbalanced CIFAR-10

What's your result of training with regular CE loss on imbalanced CIFAR-10? Tune appropriate parameters and fill in your best per-class accuracy in the table.

	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5	Class 6	Class 7	Class 8	Class 9
CE Loss	0.8030	0.9110	0.6900	0.6020	0.7670	0.7030	0.8480	0.7870	0.8900	0.8700

Table 1: Best ResNet-32 after hyperparameter tuning trained using regular CE loss on imbalanced CIFAR-10

The best precision @1 Accuracy was 0.7871 using a batch size of 64, a learning rate of 0.01129, a regularization of 5.762e-05, momentum of 0.8622, a warmup of 5, and a gamma of 1.

3.3.2 CB-Focal loss on imbalanced CIFAR-10

What's your result of training with CB-Focal loss on imbalanced CIFAR-10? Additionally, tune the hyper-parameter beta and fill in your per-class accuracy in the table; add more rows as needed.

	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5	Class 6	Class 7	Class 8	Class 9
beta= 0.999	0.7960	0.9220	0.6770	0.5880	0.7600	0.7030	0.8600	0.8050	0.8750	0.8820
beta= 0.9555	0.7750	0.9230	0.6740	0.6180	0.7880	0.7000	0.8420	0.8360	0.8950	0.8900
beta= 0.8795	0.8130	0.8980	0.7020	0.5650	0.7730	0.7180	0.8240	0.8330	0.8950	0.8820
beta= 0.7373	0.8190	0.9160	0.7010	0.5900	0.7390	0.6850	0.8490	0.8190	0.8860	0.8740
beta= 0.6410	0.8280	0.9120	0.6740	0.5850	0.7780	0.7160	0.8420	0.7980	0.8910	0.8770
beta= 0.6388	**0.8360**	**0.9220**	**0.6890**	**0.6340**	0.**7620**	**0.7170**	**0.8880**	**0.8080**	**0.8960**	**0.8870**
beta= 0.2404	0.7860	0.8720	0.6640	0.5700	0.7550	0.6850	0.8260	0.8150	0.8870	0.8830
beta= 0.1522	0.8230	0.9070	0.6940	0.5720	0.7420	0.7100	0.8110	0.8250	0.8670	0.8580

Table 2: Hyperparameter tuning of beta for ResNet-32 trained using CB-Focal loss on imbalanced CIFAR-10

The best precision @1 Accuracy was 0.8039 using a batch size of 64, a learning rate of 0.01129, a regularization of 5.762e-05, momentum of 0.8622, a warmup of 5, and a gamma of 1.

3.3.3 CE loss and CB-Focal Loss(best) together

Put your results of CE loss and CB-Focal Loss(best) together:

	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5	Class 6	Class 7	Class 8	Class 9
CE Loss	0.8030	0.9110	0.6900	0.6020	0.7670	0.7030	0.8480	0.7870	0.8900	0.8700
CB-Focal	0.8360	0.9220	0.6890	0.6340	0.7620	0.7170	0.8880	0.8080	0.8960	0.8870

Table 3: Comparing Best ResNet-32 trained with CE loss and CB-Focal Loss on CIFAIR-10

3.3.4 Analysis

Describe and explain your observation on the result: Explanation should go into WHY things work the way they do in the context of Machine Learning theory/intuition, along with justification for your experimentation methodology. DO NOT just describe the results, you should explain the reasoning behind your choices and what behavior you expected. Also, be cognizant of the best way to mindful and show the results that best emphasizes your key observations. If you need more than one slide to answer the question, you are free to create new slides.

The comparison between Cross-Entropy (CE) and Class-Balanced Focal (CB-Focal) loss on imbalanced CIFAR-10 demonstrates the effectiveness of addressing class imbalance by changing the loss function. The hyperparameters were held constant between these two models, and only the loss function was altered. The CB-Focal loss achieved a 1.7% higher overall accuracy (80.39% vs. 78.71%), with improvements in the underrepresented classes 3. The optimal beta value of 0.6388 is a moderate reweighting strength, suggesting that while class balancing is beneficial, too aggressive reweighting ($\beta > 0.9$) or too little ($\beta < 0.2$) can be detrimental to performance.

The success of CB-Focal loss can be attributed to the combination of the focal loss component ($\gamma = 1$), which reduces the contribution of well-classified examples to focus training on harder cases, while the class-balancing weight provides appropriate emphasis on underrepresented classes. The consistent improvement across most classes shows this, though some classes (like Class 2) showed minimal change.

The seemingly erratic behavior in accuracy across different β values shown in 2 can be partially explained by the relationship between β and the dataset's granularity. As explained in the original paper [6], β can be interpreted through the equation $\beta = (N - 1)/N$, where N represents the number of unique prototypes (distinct variations) within each class. For CIFAR-10, which has coarse-grained classes (e.g., "car", "bird"), each class naturally contains many unique prototypes, suggesting a larger N and thus a larger optimal β value would be appropriate.

However, the results show optimal performance at $\beta = 0.6388$, which differs from the paper's finding that CIFAR-10 performs best with larger β values (closer to 0.9999). This discrepancy might be due to differences in the imbalance factor of our dataset or other experimental conditions. The variation in accuracy across different β values is also influenced by the stochastic nature of neural network training and the complex interaction between β and other hyperparameters like learning rate and momentum. Multiple trials for each β value could be run to obtain more robust results, or a more fine-grained search around promising β values could be performed.

References

[1] Aaron Courville Ian Goodfellow Yoshua Bengio. *Deep Learning*. MIT Press, Nov. 2016. ISBN: 9780262337373, 0262337371.

[2] Justin Johnson. *Deep Learning for Computer Vision Lectures and Slides*. Lectures Slides, University of Michigan, EECS 498 Deep Learning for Computer Vision. 2019. URL: https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2019/.

[3] Robert Geirhos et al. "ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness". en. In: *arXiv* arXiv:1811.12231 (Nov. 2022). arXiv:1811.12231 [cs]. DOI: 10.48550/arXiv.1811.12231. URL: http://arxiv.org/abs/1811.12231.

[4] Zsolt Kira. *Deep Learning Lectures and Slides*. Lectures Slides, Georgia Institute of Technology, Computer Science 7643 Deep Learning. 2025. URL: https://omscs.gatech.edu/cs-7643-deep-learning.

[5] Simon J.D. Prince. *Understanding Deep Learning*. MIT Press, Dec. 2023. ISBN: 9780262377102, 0262377101.

[6] Yin Cui et al. "Class-Balanced Loss Based on Effective Number of Samples". en. In: *arXiv* arXiv:1901.05555 (Jan. 2019). arXiv:1901.05555 [cs]. DOI: 10.48550/arXiv.1901.05555. URL: http://arxiv.org/abs/1901.05555.